UNIVERSIDAD TÉCNICA DE MANABÍ
FACULTAD DE CIENCIAS INFORMÁTICAS
CARRERA: TECNOLOGÍAS DE LA INFORMACIÓN

ACTIVIDAD #1
ESTUDIO DE CASO

SISTEMA DE GESTIÓN DE RESCATE Y ADOPCIÓN ANIMAL

Estudiante: Barrezueta Parraga CLAUDIO ROBINSON

Asignatura: Desarrollo de Sistemas Informáticos

Unidad: Desarrollo de software: introducción y paradigmas de diseño

Docente: Ing. Jorge Arun Zambrano Cedeño, Mg.

Semestre: Quinto semestre

Periodo académico: 2026

1. Introducción

La atención de animales abandonados o en situación de peligro exige coordinación entre la ciudadanía, los rescatistas, el personal veterinario y los responsables de los refugios. Cuando los datos se registran de forma dispersa, puede resultar difícil conocer dónde ocurrió el rescate, quién atiende al animal y si ya está listo para ser adoptado. Por ello, se plantea una plataforma que reúna los reportes, expedientes, asignaciones, tratamientos y solicitudes de adopción en un solo sistema.

La propuesta se desarrolla como un diseño lógico, no como una aplicación terminada. Para describirla se emplean diagramas UML, principios de diseño orientado a objetos, patrones reutilizables y un flujo de trabajo Gitflow que permita registrar de manera ordenada las decisiones y los cambios.

2. Objetivos

2.1 Objetivo general

Definir la estructura lógica de una solución informática que permita administrar de forma ordenada los procesos de rescate, cuidado y adopción animal, considerando claridad, seguridad y facilidad de mantenimiento.

2.2 Objetivos específicos

Reconocer los usuarios del sistema y sus principales necesidades.

Especificar los requisitos funcionales y las cualidades de operación esperadas.

Representar las interacciones y los componentes principales mediante UML.

Proponer patrones de diseño que reduzcan el acoplamiento y faciliten futuras ampliaciones.

Organizar los documentos y cambios del proyecto con GitHub y ramas Gitflow.

3. Descripción del problema

El refugio necesita una herramienta para recibir avisos ciudadanos, confirmar los casos, registrar la información básica del animal y asignar personal responsable. Durante la atención se deben incorporar notas veterinarias y actividades de seguimiento. Una vez recuperado, el animal puede pasar a disponibilidad para adopción; la solicitud de una familia debe evaluarse y quedar asociada al expediente.

Se propone el siguiente ciclo de estados: Aviso recibido, Rescate confirmado, En evaluación, En tratamiento, Disponible para adopción, Adoptado y Caso finalizado. Los cambios de estado deben validarse y conservar fecha, usuario responsable y motivo, para facilitar consultas y auditorías.

3.1 Actores del sistema

Reportante ciudadano: comunica la ubicación aproximada, descripción y circunstancias del animal.

Coordinador de rescate: revisa reportes, prioriza casos y distribuye tareas entre los equipos.

Voluntario rescatista: consulta sus asignaciones y registra las acciones realizadas en el lugar.

Profesional veterinario: documenta evaluaciones, indicaciones y evolución del animal.

Administrador del refugio: mantiene usuarios, permisos, catálogos y supervisa el flujo de adopción.

Persona adoptante: explora animales habilitados y presenta información para ser evaluada.

4. Requerimientos del sistema

4.1 Requerimientos funcionales

RF01: Registrar un aviso con fecha, ubicación, descripción y datos de contacto autorizados.

RF02: Crear un expediente único para cada animal atendido y clasificarlo por especie.

RF03: Asignar coordinadores, rescatistas o veterinarios de acuerdo con las necesidades del caso.

RF04: Registrar intervenciones, observaciones clínicas y seguimiento del animal.

RF05: Cambiar el estado del expediente únicamente cuando se cumplan las condiciones definidas.

RF06: Enviar avisos a los usuarios autorizados cuando se crea o actualiza un caso.

RF07: Registrar solicitudes de adopción y documentar su evaluación.

RF08: Buscar y filtrar expedientes por estado, especie, fecha o responsable.

RF09: Guardar un historial de cambios y de las personas que los realizaron.

RF10: Controlar el acceso a las funciones según el rol asignado al usuario.

4.2 Requerimientos no funcionales

Seguridad: proteger los datos personales y restringir la consulta según permisos.

Disponibilidad: permitir que los usuarios autorizados consulten la información cuando la necesiten.

Mantenibilidad: separar la interfaz, las reglas del negocio y el acceso a datos.

Escalabilidad: facilitar la incorporación de nuevas especies, refugios o canales de aviso.

Integridad: validar campos obligatorios y evitar expedientes duplicados.

Usabilidad: presentar formularios sencillos y mensajes comprensibles.

5. Arquitectura lógica propuesta

La solución se organiza en cuatro capas. La capa de presentación contiene pantallas y formularios; la capa de aplicación coordina las acciones solicitadas; la capa de dominio reúne las entidades y reglas del proceso; y la capa de infraestructura gestiona persistencia, servicios de mensajería y otros recursos externos. Esta separación permite cambiar una tecnología de almacenamiento o una interfaz sin reescribir todas las reglas del sistema.

Presentación: formularios de reportes, panel de casos, fichas de animales y módulo de adopciones.

Aplicación: controladores y servicios que validan operaciones y coordinan el flujo.

Dominio: ExpedienteAnimal, Usuario, Asignacion, Tratamiento y SolicitudAdopcion.

Infraestructura: repositorios, base de datos, servicio de correo o notificaciones.

5.1 Principios de diseño

Se aplican ideas de SOLID: cada clase debe tener una responsabilidad clara; las extensiones deben realizarse preferiblemente agregando implementaciones y no modificando continuamente el código estable; las clases derivadas deben respetar los contratos de sus abstracciones; las interfaces deben ser específicas; y los servicios de alto nivel deben depender de abstracciones, no de una base de datos concreta.

6. Diagrama UML de casos de uso

El diagrama resume las acciones principales de cada tipo de usuario. Los permisos concretos deberán validarse durante la implementación, especialmente para proteger los datos de contacto y los registros veterinarios.



Figura 1. Diagrama de casos de uso propuesto.

7. Diagrama UML de clases

El modelo conceptual presenta las entidades que concentran la información principal. Usuario identifica a las personas autorizadas; ExpedienteAnimal describe el caso; Asignacion registra quién atiende el caso; Tratamiento almacena notas de atención; SolicitudAdopcion documenta el proceso de adopción; y ServicioAvisos comunica eventos relevantes.

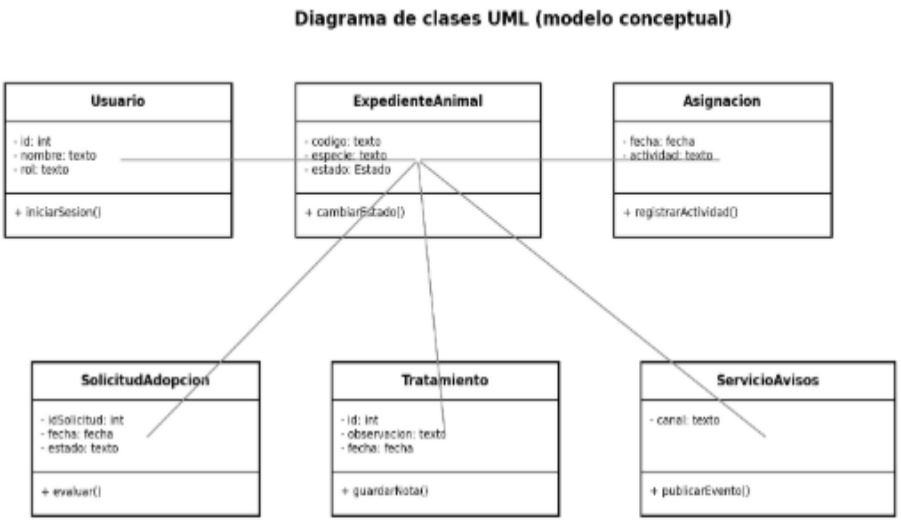


Figura 2. Diagrama conceptual de clases.

8. Diagrama de secuencia

El flujo comienza cuando una persona envía un reporte. El formulario transmite los datos al controlador, que solicita al servicio de casos validar la información. Si los datos son correctos, se genera el expediente y se guarda mediante el repositorio. Después se puede emitir una notificación a los usuarios pertinentes. En una implementación real, los errores deben informarse sin confirmar un registro que no se haya guardado correctamente.

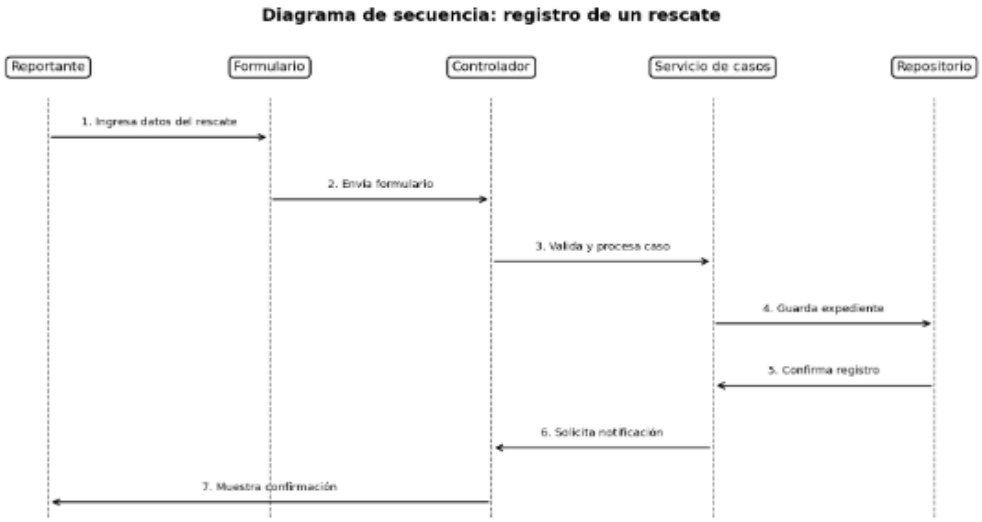


Figura 3. Secuencia de registro de un rescate.

9. Patrones de diseño propuestos

9.1 Factory Method (creacional)

Se utilizará una fábrica para crear expedientes según la categoría de animal seleccionada. La aplicación solicitará un expediente a una abstracción común y la implementación adecuada devolverá el tipo correspondiente, por ejemplo, para perro, gato o fauna silvestre. Así se evita dispersar condicionales de creación por diferentes partes del programa.

9.2 Observer (comportamiento)

El servicio de eventos publicará novedades cuando se registre un rescate, se asigne un responsable o cambie el estado de un expediente. Los suscriptores autorizados podrán recibir el aviso por los canales configurados. El servicio de casos no necesitará conocer los detalles de cada canal de comunicación.

9.3 Strategy (comportamiento)

Las reglas para priorizar un rescate pueden variar según el riesgo, la especie, el estado del animal o la ubicación. Una estrategia permite encapsular cada criterio de priorización y seleccionar el adecuado sin modificar el flujo principal de registro. Las reglas concretas deben ser definidas y aprobadas por el refugio.

9.4 MVC (organización arquitectónica)

El modelo representa los datos y reglas; la vista muestra la información y recibe entradas; el controlador interpreta las acciones y coordina los servicios. Esta división mejora la claridad del código y permite evolucionar la interfaz con menor impacto en el dominio.

10. Uso de GitHub y Gitflow

Se recomienda crear un repositorio público con el nombre sistema-gestion-rescate-adopcion. La rama main conservará las versiones estables y develop reunirá los avances integrados. Cada parte del trabajo se desarrollará en una rama feature creada desde develop y se incorporará mediante una solicitud de integración revisada.

feature/requisitos: análisis del problema y especificación de requisitos.

feature/uml: elaboración y revisión de diagramas.

feature/patrones: documentación de los patrones seleccionados.

feature/arquitectura: descripción de capas y reglas de diseño.

release/1.0: preparación de una entrega revisada, si el proyecto lo requiere.

Ejemplos de mensajes de commit: “docs: documentar actores y requisitos”, “docs: incorporar diagramas UML”, “docs: explicar patrones de diseño” y “docs: describir estrategia Gitflow”. Estos mensajes son ejemplos; para demostrar el uso real de GitHub se deben crear el repositorio, las ramas y los commits, y adjuntar capturas propias del historial y de las integraciones.

11. Seguridad, pruebas y mantenimiento

El sistema deberá validar la información antes de almacenarla, aplicar permisos por rol y mantener un registro de las operaciones relevantes. Los datos personales se usarán

únicamente para los fines autorizados. Las pruebas recomendadas incluyen: campos obligatorios, duplicidad de expedientes, transiciones permitidas entre estados, asignación de responsables, registro de tratamientos, evaluación de solicitudes y emisión de notificaciones.

12. Conclusiones

La propuesta centraliza el ciclo de atención de un animal, desde el reporte inicial hasta la adopción o el cierre del caso. Los diagramas UML ayudan a comunicar las responsabilidades y la interacción entre componentes antes de programar. Los patrones Factory Method, Observer y Strategy resuelven necesidades distintas y permiten extender el sistema con menor acoplamiento, mientras MVC y la separación por capas organizan el diseño. Finalmente, GitHub con Gitflow ofrece trazabilidad de las decisiones y facilita revisar los cambios.

13. Referencias

Atlassian. (s. f.). Gitflow workflow. https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

GitHub Docs. (s. f.). GitHub documentation. https://docs.github.com/

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1. https://www.omg.org/spec/UML/2.5.1/